THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 02 OF 07

Module 1 — Architecture Fundamentals and Design Decisions

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

02



YOUR STUDY COMPANION

American Express Solution Architect — System Design Preparation Lesson 1: Requirements → Architecture → Trade-offs

> **Scope:** This is a worked design exercise and general architecture guidance, not a description of American Express's internal architecture. Numbers, service interfaces, and workloads are illustrative assumptions.

NAVIGATION

Inside this guide

01	1. The most important principle
02	2. Understand five types of requirements
03	3. How to derive architecture decisions
04	4. Problem and requirements
05	5. Start with a simple baseline
06	6. Introduce an authoritative financial source
07	7. Introduce reliable events
08	8. Choose the database and schema
09	9. Estimate scale before choosing infrastructure
10	10. Design for failures
11	11. Security and observability
12	12. Final architecture
13	13. Change the requirements; change the architecture
14	14. Concepts and takeaways
15	Next: Design 2 — Credit Card Authorization System

CHAPTER

1. The most important principle

Architecture decisions must come from requirements, not from technology preferences.

Consider three financial applications:

System	Most important requirement	Likely architectural approach
Transfer money between accounts	Never lose or create money	ACID transactions and strongly controlled writes
View transaction history	Fast reads over large amounts of data	Read-optimized database, indexing, and pagination
Detect fraudulent transactions	Very fast decisions using current signals	Low-latency scoring, streaming, and fast feature access

All three handle financial transactions, but they need different architectures.

Weak approach: "We need a scalable system, so we'll use microservices, Kafka, Redis, Kubernetes, and NoSQL."

Better approach: "The system processes money. Correctness is critical, while some downstream updates can be delayed. Therefore, we'll use transactional writes for financial state and asynchronous events for noncritical processing."

The second approach explains *why* each technology or pattern is appropriate.

CHAPTER

2. Understand five types of requirements

Requirement	What it means	Example
Correctness	What must never go wrong?	Never debit an account twice for one logical transfer
Performance	How quickly must work complete?	Authorization must finish within a defined latency budget
Scale	How much traffic and data?	Large transaction history with high peak read traffic
Availability	What happens when components fail?	A service remains available after one instance fails
Security	Who can access what?	Users cannot read another customer's transactions

Also evaluate **cost, maintainability, compliance, and operational complexity**.

Separate requirements that cannot be violated from those where trade-offs are acceptable. For example, a history screen might tolerate a short visibility delay; a financial ledger cannot tolerate duplicate postings caused by the same business instruction.

CHAPTER

3. How to derive architecture decisions

A. SQL or NoSQL?

Start with *operations and guarantees*, not the product name.

Requirement	Likely choice	Why

Money transfers with strict invariants	Relational database	Transactions, constraints, and controlled concurrency
Simple key-based lookup at very large scale	Key-value database	Efficient access by primary key
Flexible nested documents	Document database	Flexible data structure
Complex full-text search	Search engine	Inverted indexes and relevance queries
Aggregate billions of events	Analytical database	Columnar storage and analytical query execution

Concept — ACID: Atomicity, Consistency, Isolation, Durability. ACID provides guarantees inside a supported transaction boundary; it does not automatically provide consistency across unrelated services or databases.

B. Synchronous or asynchronous?

Suppose a payment succeeds and a customer should receive an email.

Synchronous path:

```
Payment API
    |
    v
Process Payment
    |
    v
Send Email
    |
    v
Return Response
```

The email dependency can delay the response or make the request appear to fail after payment processing has succeeded.

Asynchronous path:

```
Payment API
    |
    v
Process Payment
    |
    +----> Return Response
    |
    v
Publish Durable Event
    |
    v
Email Consumer
```

The email is independent of the payment's critical path. But publishing it must still be reliable, for example through a **transactional outbox**.

Rule: Choose synchronous interactions when the caller requires an immediate answer, and asynchronous processing when the work can safely happen later or independently.

C. Strong or eventual consistency?

The displayed transaction list may lag behind the authoritative ledger if the user experience permits it. An available-credit decision, however, must use a state model that protects the required financial invariants.

- **Strong consistency:** Reads and writes satisfy a specified ordering/visibility guarantee.
- **Eventual consistency:** Replicas or read models may temporarily lag but converge when updates stop.

A single business system can use stronger guarantees for financial state and eventual consistency for history, analytics, or notifications.

D. Cache or direct database access?

Caching is valuable when read repetition is high and the business can tolerate defined staleness.

- **Good candidate:** Merchant display metadata.
- **Conditional candidate:** The first page of transaction history, if freshness requirements permit.
- **Poor candidate as sole source:** An authoritative funds-availability decision using stale cache data.

Caching creates invalidation, expiry, eviction, stampede, and outage concerns. Add it to address demonstrated needs.

E. Monolith or microservices?

A monolith can be the best baseline for a small domain with unified releases and modest scale. Microservices are more attractive when capabilities require independent releases, scaling, ownership, or security boundaries.

Microservices introduce network failure modes, distributed data management, observability needs, and operational overhead.

Principle: Prefer the simplest architecture that meets requirements; add complexity only when justified.

Worked Design 1 — Credit Card Transaction History Service

CHAPTER

4. Problem and requirements

Design a service that allows credit card customers to view recent and historical transactions, filter activity, and paginate results.

Functional requirements

Requirement	Explanation
Recent transactions	Show latest activity
Transaction details	Amount, merchant, currency, status, date
Filtering	Date and status filters
Pagination	Efficiently return successive pages
Security	Only authorized account holders/users can access data
Status updates	Handle changes such as pending to posted

Out of scope: Payment authorization, money transfers, refunds, and financial settlement. Those belong to other capabilities.

Illustrative non-functional assumptions

- The workload is read-heavy.
- The service holds large historical datasets.

- Reads should be fast and predictable.
- A small, explicitly defined visibility lag is acceptable for some activity.
- Historical records must not be lost.
- Another financial subsystem owns the authoritative financial state.

These assumptions justify considering a read-optimized projection.

CHAPTER

5. Start with a simple baseline

```
Customer App
     |
     v
Transaction History API
     |
     v
Relational Database
```

Why? The schema is structured. Queries primarily select one account's activity in date order. Indexes support those queries. We have not identified a reason to introduce Kafka, sharding, Redis, or a multi-service architecture.

Define the API

```
GET /v1/accounts/{accountId}/transactions?limit=50&cursor=opaque-value
```

Example response:

```
{
  "transactions": [
    {
      "transactionId": "txn_123",
      "amountMinor": 2599,
      "currency": "USD",
      "merchant": "Example Store",
      "status": "POSTED",
      "postedAt": "2026-11-01T14:30:00Z"
    }
  ],
  "nextCursor": "opaque-value"
}
```

`amountMinor: 2599` represents USD 25.99 in cents, rather than using an imprecise binary floating-point currency value.

Why cursor pagination instead of OFFSET?

An offset query with a very large offset may scan or skip many records. Cursor pagination resumes after a known position using an index.

```
CREATE INDEX idx_account_history
ON transactions (
    account_id,
    posted_at DESC,
```

```
    transaction_id DESC
);
```

The transaction ID is a deterministic tie-breaker when timestamps match.

Concept — Composite index: An ordered index over multiple columns. Its usefulness depends on the query's filters and sorting order.

Cursor pagination helps with deep browsing, but updates arriving between requests can still affect stability. Define how changing datasets and cursors behave.

CHAPTER

6. Introduce an authoritative financial source

A history service should consume records from the system responsible for financial transaction processing. It should not independently decide whether payments are authorized or settled.

```
Financial Processing System
            |
            v
  Authoritative Data Store
            |
            v
  Reliable Change Events
            |
            v
 Transaction History Store
            ^
            |
  Transaction History API
            ^
            |
       Customer App
```

Why separate stores? Financial processing prioritizes correctness; transaction browsing prioritizes fast read queries. Separating responsibilities prevents high-volume customer reads from directly competing with critical financial writes.

Concept — CQRS (Command Query Responsibility Segregation): Different models may handle commands and queries. They can have separate stores, but this is not mandatory. CQRS adds synchronization and consistency complexity.

CHAPTER

7. Introduce reliable events

The dual-write problem

Imagine the financial system commits a record but crashes before publishing an event.

```
1. Save financial record
2. Commit database
3. Publish event
```

A crash between steps 2 and 3 may leave the history projection permanently unaware of the transaction.

Transactional outbox

Write the authoritative change and an outbox record atomically in the *same local database transaction*:

```
BEGIN;
-- Save financial record
-- Save outbox event
COMMIT;
```

Then a publisher sends outbox records to a broker:

```
Financial Processing System
            |
            v
+--------------------------+
| Financial Database       |
|  - Transaction Records   |
|  - Outbox Events         |
+--------------------------+
            |
            v
     Outbox Publisher
            |
            v
        Event Broker
            |
            v
     History Projector
            |
            v
       History Store
```

Concept — Transactional outbox: Persist a business change and the intention to publish its event in one local transaction.

Important failure: The publisher can successfully send an event and crash before recording success. The event may then be published again. Consumers therefore must be **idempotent**.

Idempotency: Repeating the same logical operation must not create additional business effects. The consumer can track unique event IDs, enforce database constraints, and use transaction ID/version to reject duplicates or stale updates.

Do not treat broker delivery alone as an unconditional exactly-once business guarantee.

CHAPTER

8. Choose the database and schema

```
CREATE TABLE transactions (
    transaction_id VARCHAR(64) PRIMARY KEY,
    account_id VARCHAR(64) NOT NULL,
    amount_minor BIGINT NOT NULL,
    currency CHAR(3) NOT NULL,
    merchant_name VARCHAR(255),
    status VARCHAR(30) NOT NULL,
    posted_at TIMESTAMP,
    source_version BIGINT NOT NULL
);
```

Why SQL? Structured schema and predictable account/date queries make a relational store a reasonable baseline. A distributed key-value or wide-column database might be preferable if scaling and access needs justify it; NoSQL is not inherently required for a large system.

Pending vs. posted activity

Pending authorizations and posted transactions are different lifecycle states. A history API may combine a pending-activity projection with posted transactions, but must avoid double-display and avoid implying pending amounts are already settled.

CHAPTER

9. Estimate scale before choosing infrastructure

Hypothetical workload: 36 million transaction updates/day.

Metric	Illustration
Average writes	~417/s
Assumed 10× peak	~4,167/s
Raw records at 1 KB each	~36 GB/day
Raw data for 365 days	~13.1 TB

These estimates exclude indexes, replicas, metadata, audit data, and backups. Read rates need separate assumptions.

Derive scaling actions from observed bottlenecks:

Observed bottleneck	Consider	Reason
API CPU saturation	Add stateless API instances	Horizontal scale
Slow history queries	Optimize query and indexes	Fix access path before adding systems
Repeated expensive reads	Cache eligible queries	Reduce redundant database load
Read pressure on primary	Read replicas or read-specific store	Isolate reads
Hard-to-manage historical volume	Partition/archive records	Manage large datasets efficiently
One database cannot meet capacity	Sharding	Distribute load and storage

Partitioning: Organize data into subsets, possibly within a database, such as by time. **Sharding:** Distribute data across independent database nodes; adds routing, rebalancing, and operational complexity.

Since most history requests target one account, account-based distribution could work, subject to hotspot and reporting requirements.

CHAPTER

10. Design for failures

Failure	Desired behavior	Design response
API instance crashes	Other instances serve requests	Load balancer and stateless replicas
Database is unavailable	Controlled failure or approved fallback	Failover and bounded retries

Broker goes offline	Financial event is not lost	Durable outbox and eventual replay
Consumer crashes after processing	Duplicate event causes no duplicate effect	Idempotent consumer
Events arrive out of order	Older state never overwrites newer state	Versions and valid state transitions
Projector falls behind	Detect delayed history freshness	Lag metrics, alerts, replay
Popular queries overload DB	Protect storage performance	Query limits, rate limiting, caching
Region goes offline	Recover to business targets	Replication and disaster recovery plan

Timeout: Stop waiting after a defined interval. A timeout does *not* prove the remote operation failed.

Retry with exponential backoff: Retry eligible transient failures after increasing delays, with jitter and retry limits. Operations must be safe to retry.

Circuit breaker: Stop invoking a repeatedly failing dependency temporarily so that failures do not cascade.

CHAPTER

11. Security and observability

Security:

- Authenticate callers and authorize access to the specific account on every relevant request.
- Do not trust a customer-supplied `accountId` by itself.
- Encrypt data in transit and at rest.
- Use least-privilege service access and controlled credentials.
- Mask sensitive identifiers and maintain access audit trails.
- Apply appropriate retention and privacy requirements.

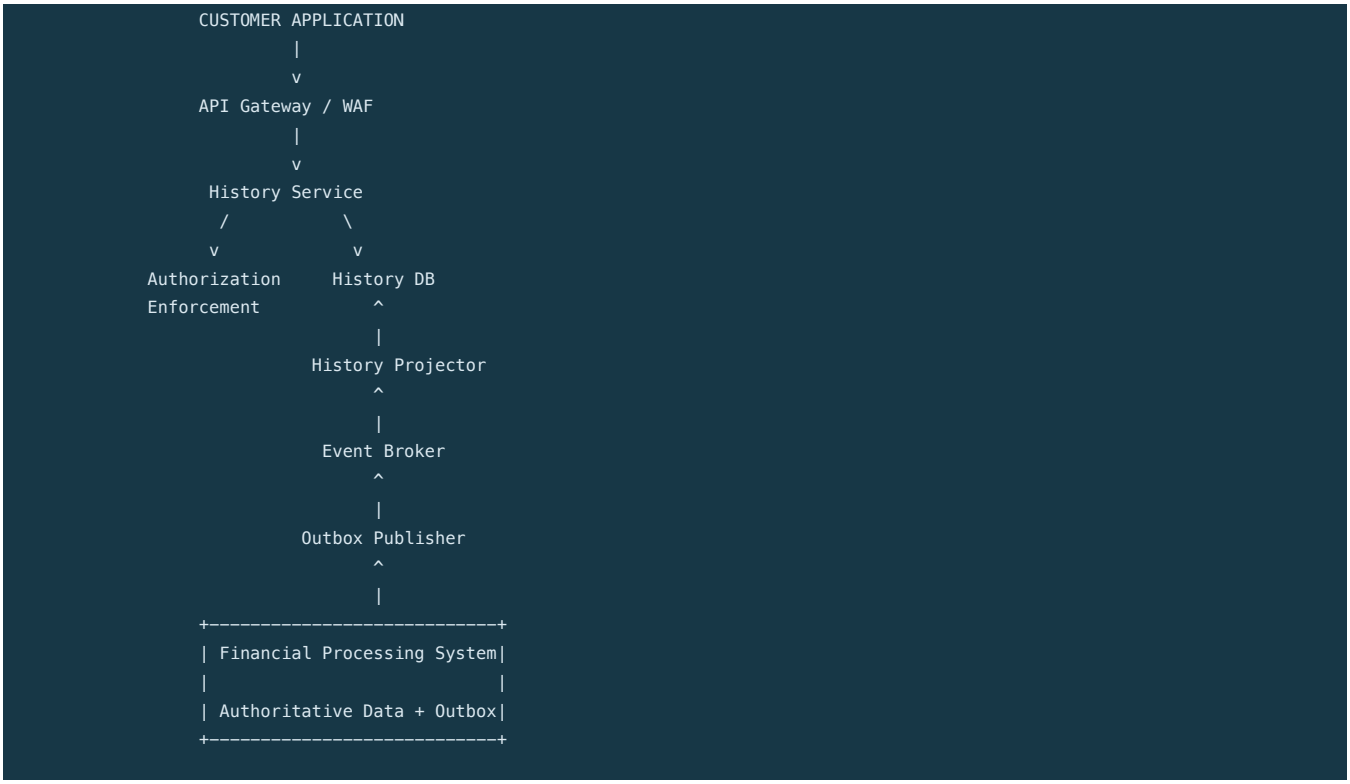
Observability:

Metric	Purpose
p95/p99 API latency	Detect slow tail requests
Error rate by endpoint	Detect customer-facing failures
Consumer lag	Measure projection delay
Oldest pending outbox event	Detect publication backlog
DB query latency	Diagnose access paths and saturation
Reconciliation mismatch count	Find disagreement between source and read model

p99 latency: 99% of measured requests finish at or below this response time, under the stated measurement window and traffic distribution.

CHAPTER

12. Final architecture



The history projection serves optimized customer reads, while the financial system remains the source of truth. Additional replication, caching, partitioning, and sharding are conditional on explicit needs.

CHAPTER

13. Change the requirements; change the architecture

New requirement	Change	Why
Show authoritative available balance	Query authoritative ledger/balance service	History may lag
Advanced merchant-name search	Add search index	Specialized full-text access
Prevent silent changes to financial history	Immutable ledger entries and compensating corrections	Preserve accounting auditability
Regional disaster recovery	Tested replication and recovery	Meet stated RPO/RTO
Notify customer of new transactions	Notification event consumer	Separate side effects from history reads
Real-time fraud decision	Low-latency scoring in authorization path	A lagging history projection is insufficient

CHAPTER

14. Concepts and takeaways

Concept	Core lesson
Relational databases	Choose around query patterns and transaction guarantees

Indexes / cursor pagination	Optimize actual API access patterns
CQRS	Separate read and write models when justified
Events	Decouple downstream work from critical processing
Transactional outbox	Prevent loss between database commit and event publication
Idempotency	Retries and duplicate delivery must not duplicate business effects
Eventual consistency	Accept lag only where the business permits it
Scaling	Measure before introducing extra complexity
Resilience	Design for partial failures and uncertain outcomes
Security	Enforce object-level authorization and protect data

Central lesson: A business system can use several consistency models, storage strategies, and communication styles. Each should be selected for a particular requirement—not because it is fashionable.

CHAPTER

Next: Design 2 — Credit Card Authorization System

We will derive the architecture for the authorization path, including request validation, fraud checks, available credit, concurrency, timeout budgets, duplicate prevention, reversals, uncertain outcomes, high availability, security, and recovery.

The central contrast is that a read-optimized, eventually consistent transaction-history projection must **not** be treated as the authoritative source for approving financial transactions.